

접어서 넣어라, 흘러 읽지 말고: 소비자 기기 LLM 추론을 위한 상주(常駐) 최적화로서의 순환 깊이

Fold, Don't Stream: Recurrent Depth as a Storage-Residency Optimization for Consumer-Device LLM Inference

안기점 (Gigyeom Ahn)

독립 연구 — AI 연구 조수와의 협업으로 수행

2026년 7월 24일 — 프리프린트 v2.1, 적대적 리뷰·교차 세션 문헌 검증 반영 개정판 (영문판의 한국어 해석본)

초 록

언어 모델이 소비자 기기의 RAM을 초과하면 디코딩은 저장 절벽(storage cliff)으로 떨어진다: 토큰 하나를 생성할 때마다 활성 파라미터 전체를 디스크에서 다시 읽어야 한다. 우리는 이 절벽을 Apple M1 Max(32GB)에서 직접 실측했다: 4비트 dense 70B 모델은 생성 토큰당 약 44GB를 SSD에서 읽으며 약 0.02 tokens/s로 디코딩된다. 본 논문은 새로운 순환 메커니즘을 제안하지 않는다 — 순환 깊이(recurrent depth)(작은 층 스택의 R회 반복 실행)는 Universal Transformers 이래 확립된 기법이다. 우리의 기여는 재측정과 재프레이밍이다: 루프를 파라미터 효율이나 추론력이 아닌 저장-상주 지렛대로 다루고, 루프형 MoE 내부의 미측정 설계 축을 검토하며, 이 메커니즘이 제거하는 RAM-초과 디코드 절벽을 직접 계측한다. 30M 파라미터 통제 실험에서, 동일 상주 바이트 및 동일 학습 토큰조건의 루프 트랜스포머는 메모리를 2배로 늘렸을 때 얻는 품질의 41%(dense)~최대 54%(Mixture-of-Experts)를 회복했으며, 이 결과는 두 하드웨어/정밀도 스택에서 교차 재현되었다; 연산량 일치 대조 실험은 이 이점이 우리 규모에서 동일-토큰 체제에 한정됨을 보인다. 루프형 MoE에서 첫 통과와 expert 선택을 재사용하는 sticky 라우팅은 시드 전반에서 자유 재라우팅과 통계적으로 구별 불가능하며($\Delta = +0.003 \pm 0.004$ nats, $n=3$) expert 접촉을 top-k에 동결한다: 루프 간 라우팅 자유는 측정 가능한 어떤 것도 사주지 않으므로, 접근 지역성은 공짜다. R 스위프는 수확 포화를 보인다(접근선 약 53~55%; 실용 최적 R=3~4). 시스템 측에서는 llama.cpp에 대한 약 300줄의 그래프 접기 패치로 같은 노트북에서 절벽 제거를 실증한다: 70B dense는 0.02에서 2.40 ± 0.10 tokens/s로(약 125배), 118B-A8B MoE는 1.4에서 2.37 ± 0.42 로(약 1.7배) 가속되며 스위프는 0이다. 통제된 종착점으로서 프로덕션 루프 모델(Nanbeige4.2-3B)의 펼친 쌍둥이(flattened twin) — 층을 복제해 동일 함수를 계산하는 체크포인트(그리디 출력이 토큰 단위로 일치) — 를 구축했다. 접힌 형태(4.4GB)는 들어가지만 펼친 형태(7.8GB)는 들어가지 않는 메모리 예산 아래에서, 쌍둥이는 0.1 tokens/s로 붕괴하고(SSD 읽기 231GB) 접힌 형태는 15.1 tokens/s를 유지한다 — 품질이 구성상 동일한 151배 격차로, 상주만이 유일한 원인임을 분리해낸다. 같은 프로덕션 모델은 이 기계에서 일관된 출력으로 49 tokens/s에 동작하며(다른 스택·규모; §4.3), 루프 네이티브 모델이 이미 이 체제를 점유하고 있다는 것과 정합적이다.

1 서론

이 연구는 실패 보고서에서 시작되었다. 32GB 통합 메모리의 M1 Max에서 Qwen3.5-122B-A10B(4비트, 73GB)를 서빙하려 시도하며 우리는 소프트웨어 도구 상자를 소진했다 — 프리페치 심도 조절, mlock 페이지 고정, 명시적 expert 캐시, 세 종류의 투기적 디코딩, expert 수 축소, 스레드 스위프 — 어느 것도 디코드 속도를 1.5 tokens/s 너머로 움직이지 못했다. 라우터 계측이 이유를 설명했다: 256개 토큰을 생성하는 동안 전체 (expert, 층) 슬롯의 98%가 접촉되었고, 따라서 "핫셋"은 풀 전체이며 어떤 캐싱 정책도 도움이 될 수 없다. 병목은 연산이 아니라 토큰당 바이트다: 자기회귀 스텝은 활성 파라미터 전체를 읽어야 하며, 모델이 RAM을 초과하는 순간 그 읽기는 수십~수백 GB/s의 메모리 대신 약 2GB/s의 저장장치를 때린다.

수학적 탈출구는 둘뿐이다: 덜 읽거나(최소화), 같은 바이트를 여러 번 읽거나(상각). 본 논문은 후자의 깊이-축 버전을 전개한다: 순환 깊이(루프) 트랜스포머는 L개의 유일한 층을 저장하고 이를 R회 실행하므로, 상주 풋프린트는 L층분 이면서 유효 깊이는 L·R이 된다. 루프 트랜스포머에 관한 선행 연구는 그 이득을 파라미터 효율[2,3]이나 테스트타임 추론[4,5,6]으로 프레이밍했다; 우리가 찾을 수 있었던 유일한 상주 논거는 MobileLLM[2]의 20MB SRAM에 관한

한 단락이다. 이 주장의 DRAM 대 SSD 버전 — 유일 가중치가 RAM에 들어가도록 스택을 접으면 저장 절벽이 사라진다 — 은 설계·실측된 적이 없어 보인다.

기여. (1) 사전 등록된 kill-gate를 갖춘 30M 규모 통제 품질 실험: 동일 상주 바이트·동일 학습 토큰에서 루프는 2배 메모리 품질 격차의 41%(dense)/최대 54%(MoE)를 회복하며, M1 Max/fp32와 NVIDIA T4/fp16에서 재현된다(§3) — 연산량 일치 대조가 이 주장을 동일-토큰 체제로 한정한다(§3.2). (2) 기존에 측정되지 않은 MoE 설계 축인 루프 내 라우팅 정책과, sticky 라우팅이 추가 expert 트래픽 0으로 자유 재라우팅과 시드 잡음 안에서 대등하다는 멀티시드 발견(§3.3). (3) 수확 포화와 실용 최적점 R=3~4를 확립하는 R 스위프(§3.4). (4) 시스템 증거: 기존 체크포인트를 추론 시점에 접는 소규모 llama.cpp 패치로 소비자 노트북에서 저장 절벽을 제거 — dense 70B 약 125배, 118B MoE 약 1.7배, 공격적 양자화 베이스라인과 병치 — 하고, 프로덕션 루프 모델의 펼친 쌍둥이 실험: 상주만으로 15.1과 0.1 tokens/s가 갈리는 동일-함수 쌍의 시연(§4).

범위. 품질 결론은 TinyStories 데이터의 30M 규모에서만 주장한다; 루프를 전제로 학습되지 않은 체크포인트를 접으면 출력 품질은 설계상 파괴되며, 우리의 접기 실험은 사용 가능한 품질이 아니라 처리량 물리를 측정한다. 두 반쪽

은 합쳐진다: 루프 학습 모델이 품질을 공급하고(\$3, [6,16]), 접기는 그런 모델을 소비자 하드웨어에서 매력적으로 만드는 상주 산술을 공급한다(\$4). 펼친 쌍둥이 실험(\$4.4)이 두 반쪽 사이의 간극을 닫는다 — 일관된 출력, 루프 네이티브, RAM 초과를 단일 측정 안에서.

2 관련 연구

순환 깊이. 층 재사용은 Universal Transformers[1]와 ALBERT로 거슬러 올라간다. MobileLLM[2]은 서브-밀리언 온디바이스 모델에 인접 블록 공유를 채택했고, Relaxed Recursive Transformers[3]는 사전학습 모델을 루프별 LoRA와 함께 재귀형으로 변환하며, Huginn[4]은 깊이 64까지 루프해 테스트타임 컴퓨트를 확장하고, Mixture-of-Recursions[5]는 토큰별 재귀 깊이를 라우팅하며, Ouro[6]는 7.7T 토큰으로 루프 모델을 사전학습해 1.4B×4가 dense 4B와 대등함을 보였다. MELT[7]는 루프 간 KV를 공유한다. 루프형 MoE 자체도 확립되어 있다: MoEUT[18]는 공유층(Universal Transformer) 형태에 FFN-어텐션 모두 MoE를 결합했고, LoopMoE[19]는 반복-조건 변조와 파라미터-FLOP 일치 루프-대-일반 MoE 비교를 더했으며, Hyperloop Transformers[20]는 middle 블록만 루프하고 — 우리 프로토클과 같은 begin/middle/end 분할(\$3.1) — 그 이점이 사후 양자화에서도 유지됨을 보고한다; LT2[21]는 루프 내부의 어텐션을 선형 변형으로 대체한다. 이 계보 전체에서 명시된 이득은 파라미터, 품질, 또는 추론력이며, 저장 I/O는 측정되지 않는다.

Tied experts. 우리의 §3.3에 가장 가까운 Tying the Loop[8]은 연속 층에 걸쳐 expert FFN 가중치를 공유하되 층별 독립 라우터를 사용하며, attention을 풀어주면 tied 그룹 내 완전 재라우팅이 거의 무비용임을 보였다 — 우리의 re-route 조건과 일치하는 결과다. 그러나 이 연구는 라우팅을 루프 간 고정하는 것을 검토하지 않고, expert 접착 수나 대역폭도 측정하지 않으며, 프레이밍은 층 파라미터 절감이다. 우리의 sticky 대 re-route 비교와 접근 지역성 결과가 그 공백을 차지한다.

저장 계층 추론. LLM-in-a-flash[9], AirLLM류 층 스트리밍, FlexGen[10]은 *비루프* 모델의 가중치를 작은 메모리로 흘려보낸다; MoE 오프로딩 시스템[11,12]은 라우팅 지역성을 활용하지만, 우리 측정에 따르면 기본 모델은 그런 지역성을 신뢰할 만큼 제공하지 않는다. 어느 것도 스트리밍과 층 재사용을 결합하지 않는다. SHARP[13]은 인접 층 쌍을 공유해 iPhone에서 저장 용량과 로딩 시간 42% 절감을 보고한다 — 메커니즘상 인접하지만 루프가 아닌 쌍 공유이고, 그 모델들은 RAM에 들어가므로 RAM-초과 절벽 자체가 쟁점이 아니다. 2024년의 llama.cpp 논의[14]는 런타임 층 재사용을 제안했으나 구현 측정되지 않았다. Nanbeige4.2-3B[16]은 우리가 아는 한 최초의 프로덕션 루프 릴리스(22층×2)이며 파라미터-없는-용량으로 프레이밍된다; 우리는 이를 생태계 검증에 사용한다.

따라서 우리의 주장은 좁고, 다각도 조사(시스템 학회, [2~6, 18~21]의 인용 그래프, 특히, 커뮤니티)의 범위 안에서 비어 있다: 루프 × RAM 초과 × 저장 I/O 실측의 3중 결

함. 루프 트랜스포머 영역이 이제 불빈다는 사실 — 공유층 MoE[18,19], 부분 루프 구조[20], 루프 효율 변형[7,21] — 은 이 관찰을 약화시키기보다 날카롭게 한다: 이들 중 누구도 저장 I/O, RAM-초과 디코드 처리량, 루프 내 라우팅 정책을 측정하지 않는다. 이 문장의 프레이밍 위험을 명시적으로 표기해 둔다: 좁은 조건 셋의 교집합은 구성상 자명하게 비어 있으므로, 그 공백 자체는 기여의 증거가 아니다. 무엇이 새롭고 무엇이 아닌지 정확히 하면 — 우리는 새 메커니즘이 아니라 측정과 프레이밍의 기여를 주장한다. 층 재사용, 루프 간 KV 공유[7], 저장 계층 스트리밍[9~13]은 각각 개별적으로 확립되어 있다; 우리가 더하는 것은 (i) 실제 체크포인트에서 루프 하의 RAM-초과 디코드 절벽에 대한 최초의 직접 계측 — 동일-함수 대조 실험 포함 — (§4), (ii) 루프형 MoE 내부의 미측정 설계 측(\$3.3), (iii) 이 결합이 함의하는 상주 대 컨텍스트 트레이드오프(\$5, 그림 4)다. "3중 교집합이 비어 있었다"를 불충분한 근거로 여기는 독자는 (i)~(iii)을 대신 저울에 올리면 된다.

3 통제 품질 실험

3.1 프로토콜

단일 GPT 구현이 설정만으로 모든 변형을 표현한다 (middle-cycle 공유: 첫/끝 층 비공유 — Hyperloop Transformers[20]도 쓰는 begin/middle/end 분할; 입력 재주입; 루프 인텍스 임베딩; pre-norm RMSNorm). 데이터는 4k BPE 어휘의 TinyStories(학습 4.807억 / 검증 480만 토큰); 학습은 AdamW(6e-4, cosine), 유효 배치 32,768 토큰, 고정 시드로 모든 조건이 동일한 배치 순서를 소비한다. 지표: 검증 손실; MoE는 추가로 (토큰, 층-슬롯)당 접착한 유니크 expert 수의 평균(루프 누적). kill-gate는 실행 전에 고정했다. **회복률** = (small - loop)/(small - large): 상주 메모리를 2배로 늘렸을 때 얻는 품질 중 루프가 공짜로 얻는 비율로 정의한다.

3.2 Dense: 동일 상주 비교

각 2억 토큰으로 세 모델을 학습했다: *small*(4층, 깊이 4, fp16 상주 30.4MB), *loop*(같은 4층의 중간 블록을 ×3 루프, 깊이 8, 30.4MB), *large*(8층, 깊이 8, 55.9MB). 표 1이 보이듯 루프는 동일 상주 기준선을 명확히 이기고 2배 격차의 약 41%를 회복하며, 실험 전체가 독립된 하드웨어-정밀도에서 재현되었다. 민감도 대조는 통과했고(*large-small* = 0.091 nats), 루프 학습은 기준선 대비 1.9배의 시간이 들었다 — 연산으로 지불하는 예상된 대가다.

그 대가는 진짜 교란 변수다: 루프 조건은 동일-토큰 기준선보다 학습 FLOPs를 더 쓰므로, 회복된 격차의 일부는 "루프 자체"가 아니라 "더 쓴 연산"의 효과일 수 있다. 우리는 연산량 일치 대조를 실행했다: *small* 아키텍처를 루프 조건의 근사 FLOP 예산(4억 토큰, 약 2배)까지 처음부터 학습하면 검증 손실 1.4936에 도달한다 — *loop*(1.5577)는 물론 2억 토큰의 *large*(1.5038)보다도 낫다. 이 규모와 데이터 체제에서, 데이터가 아니라 연산이 병목일 때는 그 연산을 상주-크기 모델의 추가 토큰에 쓰는 편이 루프보다 낫다. 따라서 본 절의 품질 주장은 동일-토큰 결과다: 루프는 토큰 예산이 고정된 상황(데이터 제한-에폭 제한)에서 남는

연산을 회복 품질로 바꾸며, 문헌의 대규모 루프 결과[6]는 우리 예산을 훨씬 넘겨 학습한다. 이 교란은 학습 측에만 있다: 디코드 시점에는 본 논문이 겨냥하는 대역폭·병목 하드웨어에서 추가 루프 FLOPs가 거의 공짜다 — §4의 실측(스트리밍 0.02 tokens/s, 토큰당 약 44GB의 SSD 읽기로 포화, 연산 아님)이 디코드 FLOPs는 교환 대상 병목이 아님을 보이므로, 상주/처리량 주장(§4)은 영향받지 않는다.

표 1. Dense 동일 상주 실험 (검증 손실, 2억 토큰). 두 독립 스택이 소수 3~4자리까지 일치; 데이터 파이프라인은 토큰 수 자릿수까지 일치. *small*의 소수 4자리 정확 일치는 독립된 두 런(다른 하드웨어·정밀도· RNG)의 반올림 우연이지 복사된 수치가 아니다; 나머지 두 행은 넷째 자리에서 갈리며, 이는 같은 최적점으로 수렴하는 독립 스택에서 기대되는 바다.

모델	상주	깊이	M1 Max fp32	Colab T4 fp16
small	30.4 MB	4	1.5950	1.5950
loop (R=3)	30.4 MB	8	1.5577	1.5573
large	55.9 MB	8	1.5038	1.5045

3.3 MoE: 루프 내 라우팅 정책

실제 대형 모델은 MoE이며, MoE를 루프시키는 것은 대역폭에 직결되는 질문을 낳는다: 블록이 다시 실행될 때 라우터가 expert를 다시 고르는가(re-route), 첫 통과와 선택을 재사용하는가(sticky)? 재라우팅은 토큰당 expert 합집합 — 우리의 122B 측정에서 캐싱을 죽인 바로 그 양 — 을 키울 수만 있는 반면, sticky는 이를 top-k에 동결한다. 42M 파라미터 MoE 4중(16 expert, top-2, d=384, 1억 토큰)을 학습했다: 무루프 기준선, R=3의 두 정책, 그리고 2배 메모리 상한.

표 2. 동일 상주 바이트의 루프형 MoE (최저 검증 손실; 접축 = (토큰, 층-슬롯)당 유니크 expert 평균, 루프 누적).

모델	검증 손실	접축	회복률
moe-small (무루프)	1.7655	2.000	—
moe-loop, sticky	1.7129	2.000	49.7%
moe-loop, re-route	1.7081	2.369	54.2%
moe-large (2×)	1.6596	2.000	100%

발견은 셋이다(그림 3). 첫째, 루프는 MoE에서 dense 못지 않게 — 오히려 더 잘 — 작동한다(회복률 49.7~54.2% 대 41%). 둘째, 정책 비교. 표 2의 단일 시드 런에서 sticky는 손실에서 re-route에 0.28% 뒤지지만(사전 등록 폐기 임계값: 2%), 단일 시드의 결정론은 통계적 강건성이 아니므로 n=3 시드로 비교를 반복했다: sticky-re-route 차이는 $+0.0032 \pm 0.0035$ nats — 통계적 동물이다. 초고는 표 2를 "sticky가 re-route 이득의 92%를 유지한다"로 읽었으나, 그 프레이밍이 정량화한 격차 자체가 시드 잡음 안에 있으므로 이를 폐기하고, 더 단순하고 강한 진술로 대체한다: **n=3에서 루프 간 라우팅 자유는 측정 가능한 어떤 것도 사주지 않으므로, 대역폭-최적 정책 — 접축을 top-k에 동결한 sticky — 는 공짜다.** 루프 대 기준선 격차(약 0.055 nats)는 시드 잡음보다 한 자릿수 크며 유지된다. 셋째, 자유 재라우팅조차 가능한 6개 중 2.37개만 접축한다: 학습된 라우터는 루프 간 약 81% 자기 일관적이며, 이는 라우터를 완전히 제약해도 감지 가능한 비용이 없는 이유를 시사한다. [8]과

정합적이다 — 그들의 층별 독립 라우터는 우리 re-route 조건의 유사물이다. 루프형·공유 expert MoE 구조는 존재하지만[8,18,19] 루프 내 라우팅 정책을 비교하거나 expert 접축을 계측한 것은 없다 — sticky 측과 접축 지표는 우리가 아는 한 여기서 처음 측정되었다. 용어 주의: [12]의 sticky는 토큰 측 라우팅 일관성(인접 토큰 간 expert 전환을 벌하는 학습 손실)을 뜻하고, 우리의 sticky는 추론 시 루프 측 일관성이다. 두 측은 직교하며 합성 가능하다.

3.4 R은 어디까지 가는가

$R \in \{1,2,3,4,6\}$ 을 각 2억 토큰으로 스윙했다(그림 1): 회복률은 $0 \rightarrow 27.6 \rightarrow 41.7 \rightarrow 48.5 \rightarrow 51.7\%$ 로 오르되 단계당 증분이 대략 반감한다(27.6, 14.1, 6.8, 이후 1.6/단계). 수확은 약 53~55%를 향해 포화한다; 로그 성장 가설은 기각되며 실용 최적점은 R=3~4다(R=6은 연산 40%를 더 내고 +3.2점을 얻는다). 이 천장은 용량 직관과 맞는다: 루프는 순차 깊이를 살 뿐 파라미터 저장을 사지 않으며, 지식 용량은 유일 가중치의 것으로 남는다[15]. 격차의 남은 절반에 닿으려면 지식 축(더 많은 유일 파라미터, 검색, 메모리층)이 필요하다 — 루프와 그 기법들은 경쟁이 아니라 결합의 관계다. (§3.2의 연산량 일치 단서는 스윙 전체에 적용된다: 이들은 동일-토큰 비교다.)

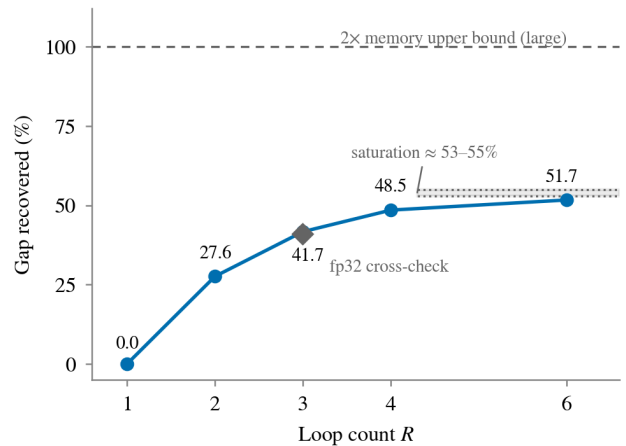


그림 1. 루프 수 R에 따른 2배-메모리 품질 격차 회복 비율 (dense, 30M 규모, 런당 2억 토큰). 증분이 단계마다 반감하며 53~55% 근처로 포화한다. R=3 점은 독립 fp32 스택에서 교차 확인(다이아몬드).

4 시스템: 저장 절벽의 제거

4.1 llama.cpp 그래프 접기 하네스

상주 주장을 실제 체크포인트에서 측정하기 위해 llama.cpp를 패치했다(두 아키텍처에 걸쳐 약 300줄): $LLAMA_FOLD_K=K$ 이면 빌드 루프는 논리 깊이를 유지하되 각 층의 가중치를 물리층 $il \bmod K$ 에서 가져온다. KV 캐시 슬롯·위치·콜백은 논리층을 유지하고, 가중치 결합 구조(헤드 수, dense 대 MoE 분기)는 물리층을 따르며, 슬라이딩 윈도우 위상 정렬은 assert로 강제한다(K는 SWA 주기의 배수여야 함). 파일에서 앞 K개 층의 페이지만 당으므로 지연 mmap 하에서 상주 집합이 K/n_{layers} 배로 줄어든다 — GGUF나 로더는 건드리지 않는다. 전체 상주 예산은 접기가 줄여주지 않는 KV 캐시를 포함해야 한다(KV는 논리 깊

이에 비례한다): 아래 70B 접기의 경우 가중치 21.2GB + KV 약 1.3GB(ctx 4096, fp16) + 버퍼 약 1GB $\approx$ 23.5GB 대 가용 약 24~26GB — 접기 배수는 이 총합이 닫히도록 선택한다. macOS에서의 측정 위생은 만만치 않았다(--no-repack 필수; 로드 프리페치 비활성; 위치독과 스왑-델타 가드); 실패 양상은 부록 A에 기록한다.

4.2 결과: 절벽의 실측과 제거

표 3. M1 Max 32GB에서 기존 체크포인트 접기 (CPU 8스레드; 런당 생성 32토큰, 단 Laguna 스트리밍 기준선은 8토큰; 접기 행의 품질은 의도적으로 평가하지 않음 — 루프 학습이 없는 체크포인트다). SSD 바이트는 런 전체의 페이지인 델타. 접기 행은 n=3 반복의 평균, $\pm$ 는 표본표준편차.

구성	Peak RSS	SSD 기	읽 스왑	디코드 t/s
Llama-3.3-70B Q4 (42.5GB), 스트리밍	21 GB	1298 GB	0	∞ 0.02
접기 R=2 (유일 21.2GB)	21 GB	31~33 GB	0	2.40 ± 0.10
IQ2_XXS (17.8GB), 스트리밍, 일관 출력	20 GB	30 GB	0	1.7
Laguna-S-2.1 118B-A8B (57.6GB), 스트리밍	22 GB	37 GB	0	1.4
접기 R=3 (상주 약 17GB)	15~20 GB	18~20 GB	0	2.37 ± 0.42

dense 절벽은 가혹하고 그 제거는 극적이다(그림 2): 스트리밍 시 70B 모델은 토큰마다 약 44GB를 다시 읽고 — 32 토큰에 SSD 트래픽 1298GB, 파일 크기의 31배 — 약 0.02 t/s로 디코딩한다; 유일 가중치가 RAM에 들어가도록 접으면 콜드 읽기는 한 번(약 31GB)뿐이고 디코딩은 2.40 ± 0.10 t/s로 돈다 — 환경변수 하나로 약 125배, 스왑 0. MoE 절벽은 본질적으로 완만하므로(토큰당 활성 expert만 읽는다) 접기 이득은 약 1.7배다; 거기서의 이득은 활성 풀의 완전 상주다. (초기 단발 런은 3.8 t/s로 읽혔으나, 반복 측정이 그 초과분을 페이지 캐시 상태로 귀속시킨다 — 부록 A(6) — 우리는 n=3 평균을 보고한다.) 이 CPU 전용 수치는 백엔드를 의도적으로 고정된 것이며 바닥이지 천장이 아니다 — 접힌 모델은 통합 메모리에 들어가므로 원리상 GPU 실행이 가능하다(우리의 Metal 경로는 현재 매핑된 텐서 전부를 wired로 등록하는 문제가 있어 공학 과제로 남긴다, 부록 A). 이 런들이 보여주지 않는 것을 강조한다: 접힌 출력은 비일관적이며, 이는 루프 학습이 없는 체크포인트에서 예상된 바다. 이 실험은 저장 물리를 분리 측정하며, 품질은 루프 네이티브 학습에서 와야 한다(§3, [3,6,16]).

표 3에 대한 회의적 독해는 이렇게 묻는다: 공격적 양자화만으로 70B 체크포인트를 예산 근처로 줄일 수 있는데 접기가 왜 필요한가. 우리는 그 베이스라인을 직접 실행했다: IQ2_XXS(가중치당 약 2.06비트, 17.8GB)는 평범한 SSD 트래픽(30GB)으로 1.7 t/s를 내며 출력은 일관적이다. 우리의 입장은 접기가 오늘의 기성 체크포인트에서 양자화를 이긴다는 것이 아니다 — 기성 체크포인트라면 공격적 양자화가 실용적 기본값이고, 그것만으로 가용 RAM에 도달하는 경우 접기는 더할 것이 없다. 접기의 가치는 전망적이며

직교적이다: (a) 루프 네이티브 모델(§3, §4.3~4.4)에는 약 2 비트급 품질세 없이 상주를 제공하며, 이는 비루프 모델의 사후 양자화가 같은 비트 수로 따라올 수 없는 것이다; (b) 양자화와 접기는 결합된다 — 접힌 모델 자체를 양자화할 수 있고($R \times Q$ 복리), 우리는 이 조합을 이미 답한 질문이 아니라 자연스러운 다음 측정으로 표기한다.

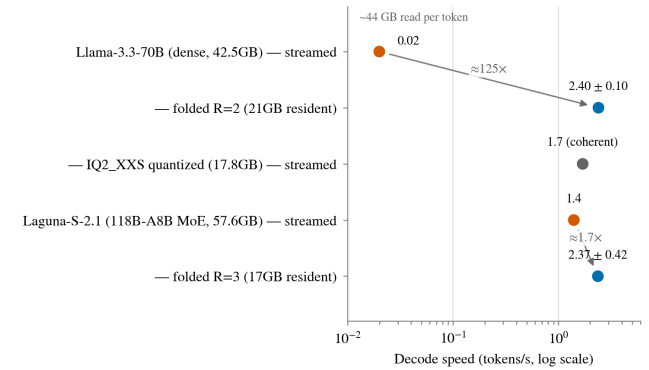


그림 2. 같은 노트북에서의 접기 전/후 디코드 속도 (로그 스케일). dense는 절벽 전체를 맞고(토큰당 약 44GB 재읽기) 약 125배를 얻으며, MoE는 활성만 읽으므로 약 1.7배를 얻는다. 공격적 양자화 베이스라인(IQ2_XXS, 회색)은 비루프 체크포인트의 실용적 선택지다: 접기보다 느리지만 출력이 일관적이다.

4.3 생태계 검증: 프로덕션 루프 모델

미학습 체크포인트 접기가 물리를 보았다면, 루프 네이티브 모델은 제품을 보인다. Nanbeige4.2-3B[16](22개 유닛 층을 2회 실행; Q4에서 2.57GB)은 벤더의 llama.cpp 브랜치로 같은 노트북에서 동작한다: 출력은 완전히 일관적이며(추론 모드의 코드 답변) Metal에서 디코드 49.4 tokens/s / 프롬프트 115 t/s를 기록한다. §3에서 30M 규모로 검증한 메커니즘은 이미 3B 규모로 출하되고 있으며, 그런 모델의 소비자 추론은 오늘 실용적이다; llama.cpp 메인라인 지원은 리뷰 중이다[17].

비교 가능성 주의: 49.4 t/s와 표 3의 2.4 t/s는 같은 축 위의 수치가 아니며 "여기서 루프가 20배 낫다"로 읽어서는 안 된다. Nanbeige는 약 3B, Q4, 루프 네이티브 체크포인트로 Metal 백엔드에서 RAM에 통째로 들어간다(2.57GB) — 그 수치는 절벽 제거가 아니라 평범한 소형 모델의 디코드 속도다. 표 3의 수치는 루프 학습이 전혀 없는 70B/118B 체크포인트를 순전히 저장 물리를 시험하려 사후에 접은 것으로, 의도적으로 고정한 CPU 백엔드다. 두 줄은 서로 다른 주장 — "루프 네이티브 모델은 이미 출하된다"와 "접기는 임의 체크포인트의 절벽을 제거한다" — 을 시연하며, 속도 순위표가 아니라 상보적 생태계 증거로서만 병치된다.

4.4 펼친 쌍둥이: 동일한 함수, 다른 상주

표 3에 대한 가장 강한 반론은 접힌 조건의 출력이 열화되어 속도와 품질이 한 번도 함께 시연되지 않는다는 것이다. 우리는 위의 프로덕션 모델로 만든 동일-함수 쌍으로 이를 닫는다. Nanbeige4.2-3B는 22개 물리층을 2회 실행한다(논리 44층); 우리는 22층을 44층 비루프 GGUF로 복제한 펼친 쌍둥이를 구축하고(Q8_0: 접힘 4.43GB, 펼침 7.78GB), 모델의 루프 경계 정규화가 두 형태에서 같은 물리 위치에 적용되도록 추론 그래프를 패치했다. Nanbeige는 루프 경계마다 출력 norm을 재사용하는 RMSNorm을 삽입하며 이는 루프 수에 게이트되어 있다; 단순 펼치기는 이를 조용히 떨어뜨려 함수를 바꾼다. 우리의 패치는 이를 설정 가능한 물리 스트라이드에서 재현해 정확한 동치

복원한다. 두 체크포인트는 같은 함수를 계산한다: 그리디 디코딩이 토큰 단위로 동일한 출력을 낸다. 제약이 없으면 둘 다 동일한 15.6 t/s로 디코딩된다(CPU 8스레드) — 이 쌍은 연산에서 전혀 다르지 않다. 이어서 mlock 벨러스트로 19GB를 고정해 가용을 약 7GB로 좁혔다: 접힌 형태는 들어가고 펼친 형태는 들어가지 않는다. 결과: 접힘 15.1 t/s(SSD 읽기 10GB); 펼침 0.1 t/s, 32토큰에 SSD 읽기 231GB, 전 구간 스왑 0 — 아키텍처·가중치·함수·출력·하드웨어가 모두 동일하고 오직 상주만 다른 151배 격차다. 이는 품질 교환을 구성상 제거한 표 3의 절벽이다: 일관된 출력의 루프 네이티브 RAM-초과 모델에서, 속도 차이는 상주 하나에 귀속된다.

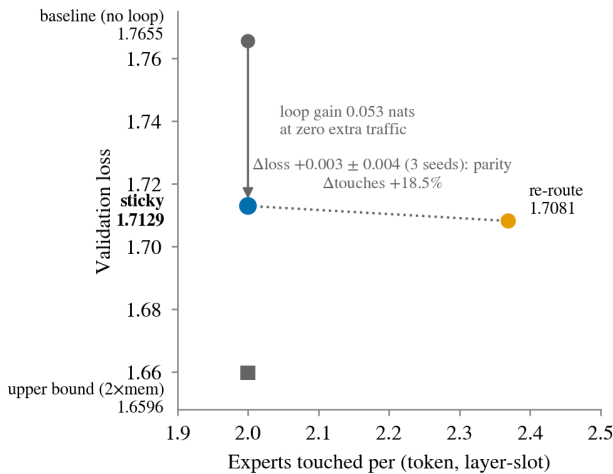


그림 3. 품질/트래픽 트레이드로 본 루프형 MoE 라우팅 정책 (§3.3). sticky는 기준선 트래픽(접촉 2,000)에 머물면서 자유 재라우팅과 통계적으로 구별 불가능한 품질을 낸다(3개 시드에서 $\Delta = +0.003 \pm 0.004$); 자유 재라우팅조차 가능한 6개 중 2.37개만 쓴다(자연 루프 간 일관성 약 81%).

5 논의와 한계

규모. 품질 결과는 TinyStories의 30M/42M 파라미터에서 확립되었다; 0.1~8B의 문헌[2,3,6]과 3B의 Nanbeige가 이전 가능성을 지지하지만, 우리 수치는 그 규모 너머를 주장하지 않는다. 우리의 토큰 예산은 루프에 보수적이며(루프는 장기 학습에서 이득이 커진다고 보고된다[6]) 따라서 회복률은 과소추정일 수 있다. TinyStories는 반대 방향으로 작용한다: 지식 요구가 낮고 구문 규칙성이 높아 — 루프가 가장 잘하는 체제라 — 같은 수치가 데이터셋 특성 쪽으로는 과대평가일 수 있다; 두 편향의 순 방향은 측정하지 않았다. **지식 천장.** 격차 절반에서의 포화는 루프가 저장이 아닌 깊이를 산다는 것과 일치한다[15]; 이득은 추론성 능력에 집중되며 지식 중심 용도는 파라미터/검색 축이 필요하다. **KV 캐시와 컨텍스트 길이.** 접기는 상주 가중치를 줄이지만 KV 캐시는 줄이지 못한다 — KV는 R과 무관하게 논리 깊이와 컨텍스트 길이에 비례한다. 그림 4가 Llama-3.3-70B(80층, GQA kv-헤드 8, 헤드 차원 128; fp16에서 토큰당 320KiB)의 체제 지도를 명시한다: 가용 24GB 예산 아래 R=2(가중치 21.2GB)는 약 9k 토큰의 컨텍스트만 유지한다; R=3은 약 31k, R=4는 약 43k에 이르고, KV를 q8로 낮추면 R=2가 약 18k로 늘어난다. 따라서 루프 간 KV 공유[7]는

선택적 다듬기가 아니다: 낮은 접기 배수(더 나은 회복 품질, §3.4)와 고정 RAM에서의 장문맥을 모두 원하는 어떤 배포에서도 성립 조건이다. 우리는 이를 평가하지 않았으며, §3의 품질 결과와 §4의 장문맥 배포 사이에 놓인 가장 큰 미해결 항목으로 표기한다. **시스템.** 측정은 한 기기(M1 Max, 한 종류의 NVMe), CPU 백엔드; Metal 상호작용은 미해결이고, 접기 품질은 의도적으로 희생되었으며, 실전 배포는 루프 학습 또는 루프 변환[3] 체크포인트를 요구한다 — 그 변환 비용(수십 B 토큰)은 우리가 지불하지 않았다. **주장 규율.** 모든 kill-gate는 사전 고정되었고, 신규성 주장은 집필 전 적대적으로 수색되었으며 그 프레이밍 위험의 인정과 함께 §2에 서술된다. 본 개정판은 적대적 리뷰를 반영한다: 멀티시드 반복 (§3.3), 연산량 일치 대조 (§3.2), 양자화 베이스라인 (§4.2), 펼친 쌍둥이 (§4.4)는 각각 리뷰가 요구했기에 존재하며, 초고의 헤드라인 주장 둘("92%"와 MoE 2.7배)은 추가 측정에 의해 폐기되거나 정정되었다. v2.1에서는 독립 리뷰 세션들의 사후 교차 검증이 원래 신규성 수색이 놓친 2024~2026 루프형 MoE 계보[18~21]와 §3.3의 토큰 축/루프 축 sticky 구분을 추가했다.

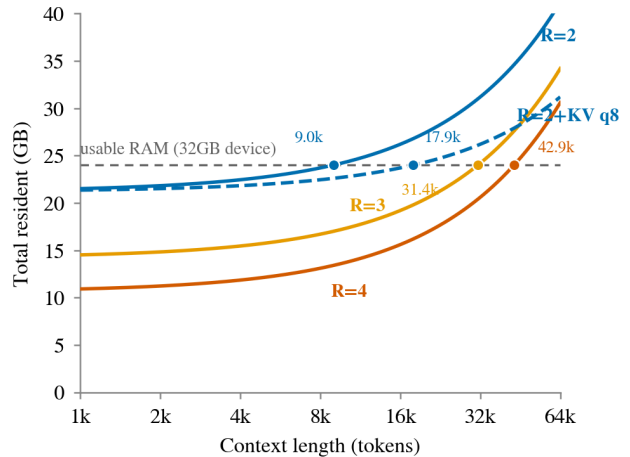


그림 4. Llama-3.3-70B의 R=2/3/4(fp16 KV)와 R=2+q8 KV에 대한 총 상주 메모리(접힌 가중치 + KV 캐시) 대 컨텍스트 길이. 마커는 32GB 기기의 가용 24GB 예산 아래 각 구성이 유지하는 최대 컨텍스트를 표시한다.

6 결론

상주 최적화로 다루면 순환 깊이는 접기 배수를 배포의 손잡이로 바꾼다: 수확이 아직 풍부한 R=3~4에서 학습하고, 접힌 스택으로 배포하고, 논리 깊이로 실행하며, 대역폭에 묶인 하드웨어에서는 sticky로 라우팅하라 — 멀티시드 데이터가 비용이 전혀 없음을 보인 정책이다. 32GB 노트북에서 이는 초당 백분의 몇 토큰짜리 저장 절벽 체제를 RAM 속도의 디코딩으로 바꾸며, 펼친 쌍둥이 실험이 그 전환을 상주 하나에 귀속시킨다: 같은 함수, 같은 출력, 151배의 속도. 첫 프로덕션 루프 모델들은 생태계가 이미 이 방향으로 움직이고 있음을 시사한다. 품질 격차의 남은 절반은 루프를 지식 축 기법(메모리층, 검색)과 결합하는 방향을 가리키며, 이는 향후 과제로 남긴다.

† 문헌 조사, 실험 오케스트레이션, 구현, 야간 실험 운영은 AI 연구 조수(Claude, Anthropic Fable 5)와 함께 수행되었다; 연구 방향, 모든 결정, 자원, 내용에 대한 책임은 인간 저자에게 있다. 모든 측정은 저자의 로

그와 로컬 저장소에서 재현 가능하다; 총 컴퓨트 비용은 GPU 크레딧 \$9.99 1회 구매와 소비자 노트북 한 대였다. 본 문서는 작업 중 프리프린트로 동료 심사를 받지 않았다. 본 한국어판은 영문판의 해석본이며 수치·주장에서 영문판이 우선한다.

참고문헌

- [1] M. Dehghani et al. Universal Transformers. arXiv:1807.03819, 2018.
- [2] Z. Liu et al. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. ICML 2024. arXiv:2402.14905.
- [3] S. Bae et al. Relaxed Recursive Transformers. arXiv:2410.20672, 2024.
- [4] J. Geiping et al. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. arXiv:2502.05171, 2025.
- [5] S. Bae et al. Mixture-of-Recursions. NeurIPS 2025. arXiv:2507.10524.
- [6] ByteDance. Ouro: Looped Language Models. arXiv:2510.25741, 2025.
- [7] V. Conchello Vendrell et al. Memory-Efficient Looped Transformer: Decoupling Compute from Memory in Looped Language Models (MELT). arXiv:2605.07721, 2026.
- [8] M. Jaggi. Tying the Loop — Tied Expert Layers in Mixture-of-Experts Language Models. arXiv:2606.16825, 2026.
- [9] K. Alizadeh et al. LLM in a Flash: Efficient LLM Inference with Limited Memory. arXiv:2312.11514, 2023.
- [10] Y. Sheng et al. FlexGen: High-Throughput Generative Inference of LLMs with a Single GPU. ICML 2023.
- [11] MoE-Infinity: Offloading-Efficient MoE Model Serving. arXiv:2401.14361, 2024.
- [12] A. Kayyam. Sticky Routing: Training MoE Models for Memory-Efficient Inference. arXiv:2607.08780, 2026; X. Zhu et al. ReMoE: Boosting Expert Reuse through Router Fine-Tuning in Memory-Constrained MoE LLM Inference. ICML 2026. arXiv:2605.27081.
- [13] SHARP: Sharing Adjacent Layers for Efficient On-Device Inference. arXiv:2502.07832, 2025.
- [14] llama.cpp discussion #4718: in-situ auto-frankenmerges (미구현 제안), 2024.
- [15] Z. Allen-Zhu, Y. Li. Physics of Language Models, Part 3.3: Knowledge Capacity Scaling Laws. arXiv:2404.05405, 2024.
- [16] Nanbeige. Nanbeige4.2-3B: a Looped-Transformer agent model. HuggingFace model card, 2026.
- [17] llama.cpp PR #25994: Add support for Nanbeige4.2 (looped transformer), 2026.
- [18] R. Csordás, K. Irie, J. Schmidhuber, C. Potts, C. D. Manning. MoEUT: Mixture-of-Experts Universal Transformers. NeurIPS 2024. arXiv:2405.16039.
- [19] W. Chen et al. LoopMoE: Unifying Iterative Computation with Mixture-of-Experts for Language Modeling. arXiv:2606.04438, 2026.
- [20] A. Zeitoun, L. Torroba-Hennigen, Y. Kim. Hyperloop Transformers. arXiv:2604.21254, 2026.
- [21] C. Deng et al. LT2: Linear-Time Looped Transformers. arXiv:2605.20670, 2026.

부록 A 소비자 하드웨어에서의 측정 위생

일곱 가지 교훈을 다른 이들이 건너뛰도록 기록한다; 앞의 다섯은 하루 저녁을 소모한 실패 양상이다. (1) llama-bench는 ARM 가중치 repack을 끝 수 없다; 42.5GB 모델의 repack은 그에 상응하는 익명 메모리를 할당해 런을 OOM으로 죽인다 — llama-cli --no-repack을 쓰라. (2) 현행 llama-cli TUI는 대화 모드를 꺼도 -n 토큰 후 종료하지 않는다; 성능 라인을 감지해 종료시키는 위치독이 필요하다. (3) 기본 로드 프리페치는 파일 전체를 건드려 상주 관측을 파괴한다; 비활성화하라. (4) Apple Silicon에서 접힌 모델의 Metal 오프로드는 매핑된 텐서 전부를 wired GPU 메모리로 등록했다(32GB 기계에 42.5GB wired) — 프로세스 RSS가 1GB로 읽히는 동안 시스템이 얼어붙었고, RSS 스왑 기반 가드는 이를 보지 못한다; 접힌 Metal 실행은 로더 수준의 버퍼 범위 지정이 필요하며 미해결로 남는다. (5) 스왑-텔타 가드(+1.5GB 초과 시 중단)와 런별 페이지인 회계가 보고된 모든 런의 무스왑을 검증 가능하게 했다. (6) mmap 기반 모델의 단발 처리량은 신뢰할 수 없다: 초기 Laguna 접기가 3.8 t/s로 읽혔으나 n=3 반복이 2.37 ± 0.42 로 정정했다 — OS 페이지 캐시는 런 사이의 숨은 변수다; 반복 평균을 보고하라. (7) 쌍둥이 실험(\$4.4)에서 RAM 제약은 VM이나 cgroup이 아닌 mlock 밸러스트 프로세스로 걸었다; 무제약 대조(두 쌍둥이 모두 동일한 15.6 t/s)가 붕괴의 원인이 펼쳐기 자체가 아니라 제약임을 확인하는 검증을 겸한다.